Aldel Education Trust's

St. John College of Engineering and Management, Palghar

NAAC Accredited with Grade A+

DEPARTMENT OF COMPUTER ENGINEERING

LLM-Based Log Anomaly Detector

with Attack Narrative and Adversarial Robustness

Harsh Bangera (Roll No. 3)

Manas Chaudhari (Roll No. 17)

Krish Mendon (Roll No. 62)

Anandan Nair (Roll No. 68)

Prof. Darshan Mhapsekar

31 August 2026

Contents

- Background of the Problem
- Problem Statement
- Motivation
- Literature Review
- Research Gaps
- Objectives
- Proposed Methodology
- System Architecture
- **Implementation and Results** *(new — the working system)*
- Technologies and Resources
- Project Schedule
- References

Background of the Problem

- Every server writes a log line for each login, process and network event. A small company produces millions a day; nobody reads them by hand.
- Security teams use a **SIEM** (Security Information and Event Management tool — Splunk, Wazuh, QRadar) that matches those lines against **written rules**.
- A rule matches **text**, not meaning. `Failed password for root` matches; the same event worded differently does not.
- Recent surveys of log anomaly detection [R2, R7] evaluate on fixed datasets where the attacker is not trying to avoid the detector.

Speaker line: "The tools in use today read the words, not the meaning."

Background: the evidence

Everything here is our own measurement on real logs, not a claim from a website.

- **Sigma rules: 0 of ~3,300 match an SSH login line.** We searched the whole open rule library for `Failed password`, `Accepted password`, `Invalid user`, `pam_unix`. – No coverage at all for the commonest attack surface on Linux.
- **Wazuh keeps 91% of a brute force — 29% after we reword it.** Same attack, same volume, different words.
- **Our detector: 97-100% of the same lines, both times.** Rewording barely moves meaning, and meaning is what we compare.
- **Alert volume on one clean day: Wazuh 4,546 alerts, us 1,415 incidents** — for 9 real attacks. Both far too many. That is analyst fatigue, measured on both sides.

Sources: `CLAUDE.md` Decision Log 2026-08-18; `results/bakeoff/bakeoff.md`; `results/killpoint/killpoint_ambient0.05.md`.

Problem Statement

- SIEM rules match the **text** of a log line, not its meaning — so an attacker who simply rewords an attack walks straight past them.
- And nobody reports **how much detection is actually lost** when the attacker is deliberately hiding, so the size of the problem is unknown.

Motivation

- **Analysts drown in alerts.** On one clean corpus a production SIEM raised 4,546 alerts for 9 real attacks. Alert fatigue is not a talking point; it is measurable.
- **Words change, behaviour does not.** An attack keeps its shape — many failures from one place, a login from two countries an hour apart — even when the wording changes. Meaning-based detection can follow that; string matching cannot.
- **It has to run where the budget is.** The whole system runs on an ordinary laptop with no GPU and no cloud account. A GPU only speeds the written explanation up (5.2 s vs 24.2 s per incident). A team with no security budget can still deploy it.

Literature Review (1 of 2)

Journal papers only — IEEE Transactions and Web of Science / Scopus indexed journals. Full PDFs are attached with this presentation.

#	Paper	Journal, Year	Method	Finding	Limitation
R1	LogFiT — Almodovar et al.	**IEEE Trans. Network & Service Management**, 2024	BERT fine-tuned on log text, self-supervised	Language models read logs without labelled data	Tested on clean benchmark logs only; no evading attacker
R2	Landauer et al. — survey of deep learning for log anomaly detection	Machine Learning with Applications (Elsevier), 2023	Survey of the field	Maps every major method and dataset	Reports that evaluation is almost always on static, non-adversarial data
R3	Ali et al. — ML techniques for log anomaly detection	Empirical Software Engineering (Springer), 2025	Classical ML vs deep learning, measured	Simpler models compete with deep models on log data	Clean data only; no attacker adapting
R4	Wu, Li & Khomh — effectiveness of log representation	Empirical Software Engineering (Springer), 2023	Compares how logs are turned into vectors	Representation choice changes results a lot; embeddings work well	Does not test reworded or disguised attacks

Literature Review (2 of 2)

#	Paper	Journal, Year	Method	Finding	Limitation
R5	Khan et al. — impact of log parsing	Empirical Software Engineering (Springer), 2024	Measures how parsing quality affects detection	Bad parsing quietly ruins detection accuracy	Parsing studied on clean logs
R6	Pang et al. — deep learning for anomaly detection: a review	ACM Computing Surveys, 2021	Field review	Standard definitions and method families	General; not security-log specific
R7	Kheddar — transformers and LLMs for intrusion detection	Information Fusion (Elsevier), 2025	2025 survey	LLMs are moving into intrusion detection fast	Evaluation still on fixed public datasets
R8	Debicha et al. — TAD: detecting evasion attacks against intrusion detection	Future Generation Computer Systems (Elsevier), 2023	Attackers actively evading a detector	Evasion measurably breaks detectors	Network traffic, **not logs**; no SIEM comparison

What all eight share: the attacker is not trying to hide from the detector being tested. R8 is the only one with a real adversary, and it is on network traffic.

Research Gaps

What exists	Gap
Rule-based SIEMs match fixed text patterns [R2, R7]	Miss the same attack once it is reworded — we measured 91% → 29%
Log anomaly models are evaluated on clean benchmark data [R1, R3, R4, R5]	Nobody reports how much detection survives when the attacker actively hides
Adversarial evasion is studied for network intrusion detection [R8]	Not for log analysis, and never against a real SIEM ruleset side by side

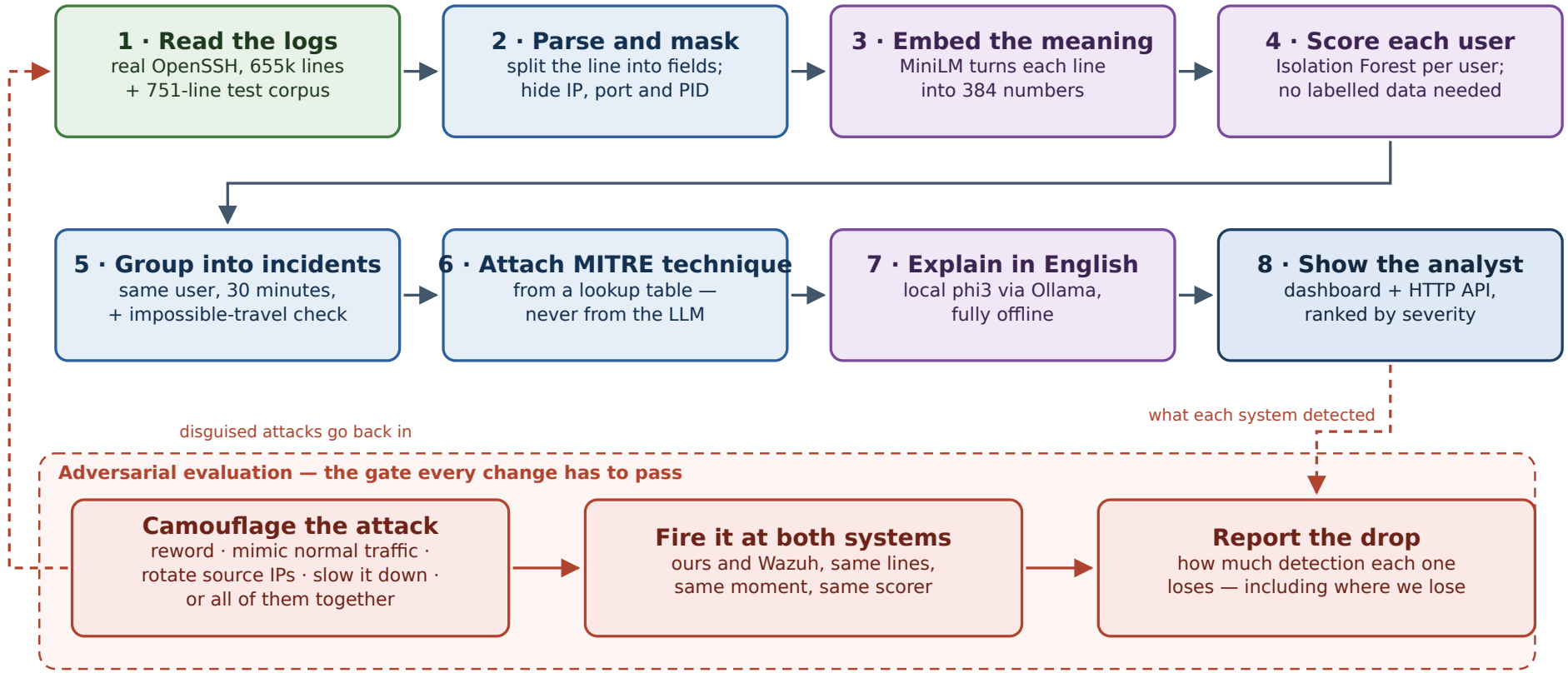
Our gap, in one line: how much worse does each system get when the attacker tries to hide — measured, on the same logs, at the same moment, for a research detector and a production SIEM together.

Objectives

- Parse raw Linux system and authentication logs into a clean, structured form.
- Detect anomalies from the **meaning** of a log line, using sentence embeddings and a per-user Isolation Forest, with no labelled training data.
- Group related anomalies into a single incident and attach the matching MITRE ATT&CK technique from a lookup table, never from the language model.
- Explain each incident in plain English with a local LLM that runs offline.
- Measure the system and a production SIEM (Wazuh) on the same logs, under eight camouflage strategies, and report the difference — including where we lose.
- Keep the whole system running on a laptop with no GPU and no cloud service.

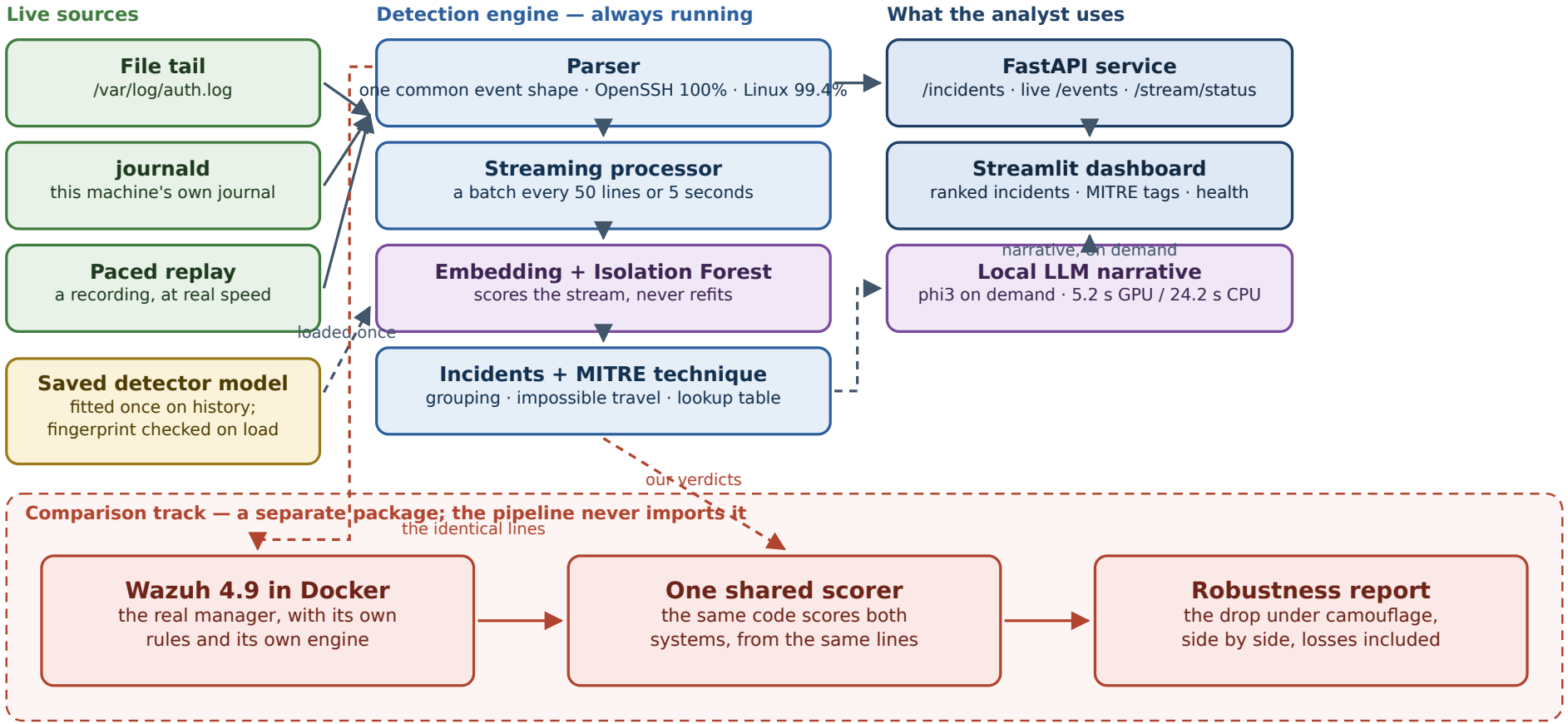
Proposed Methodology

- Read logs → parse and mask → embed the meaning → score each user → group into incidents → attach MITRE technique → explain in English → show the analyst.
- Camouflaged attacks are fed back in and both systems re-scored on every change.
- Isolation Forest over embeddings is **standard**; we do not claim it as the novelty. The measurement is the contribution.



System Architecture

- Three live sources — a tailed file, this machine's journal, or a paced replay.
- The model is fitted once on history and only **scores** the stream after that.
- Streaming produces exactly the batch result: 751 lines → 130 events → 94 incidents.
- The Wazuh comparison track is a separate package the pipeline may never import.



Implementation: keeping the comparison fair

- One writer appends **every line to both systems in the same call** — neither can be handed different traffic.
- Our injector wrote the attack, so every line's label is recorded as it goes out: ground truth is automatic.
- The **same scoring code** grades both systems.
- Built so far: foundation and 449 tests · benchmark platform with a real SIEM · live streaming, API and dashboard · the live bake-off.

Results

Robustness (5 seeds, real OpenSSH logs, both systems on the same corpus). The number is the drop in detection when the attack is camouflaged: bigger is worse.

Camouflage	Us	Wazuh	Difference
Reworded (`rephrase`)	−2.2	**−33.3**	**+31.1 in our favour**
Mimicry / noise / volume / timing	0.0	0.0	tie ×4
IP rotation	−11.1	0.0	**−11.1 against us**
Slowed down	−8.9	0.0	**−8.9 against us**
Realistic combination	−13.3	0.0	**−13.3 against us**

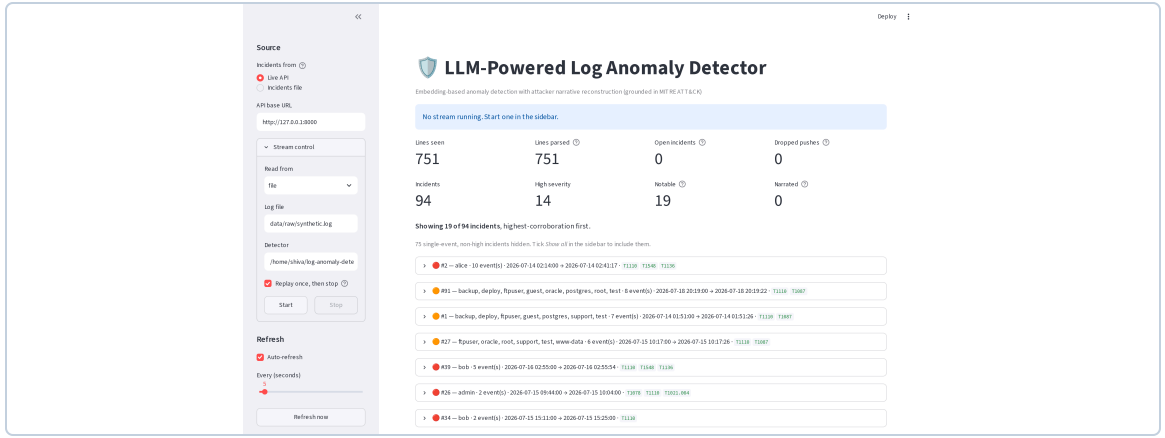
Result: 1 win, 4 ties, 3 losses. The registered threshold (≥25 points on at least one strategy, written into git before we had any result) is cleared by rewording, so the project continues — but the honest claim is narrow: *signature rules break when you reword an attack; embeddings do not.*

Live demonstration of the same effect (`results/bakeoff/bakeoff.md`, 18 conditions): rewording an impossible-travel attack took Wazuh from 3 of 3 instances to **0 of 3**, while we stayed at 3 of 3 — a 100-point gap, live.

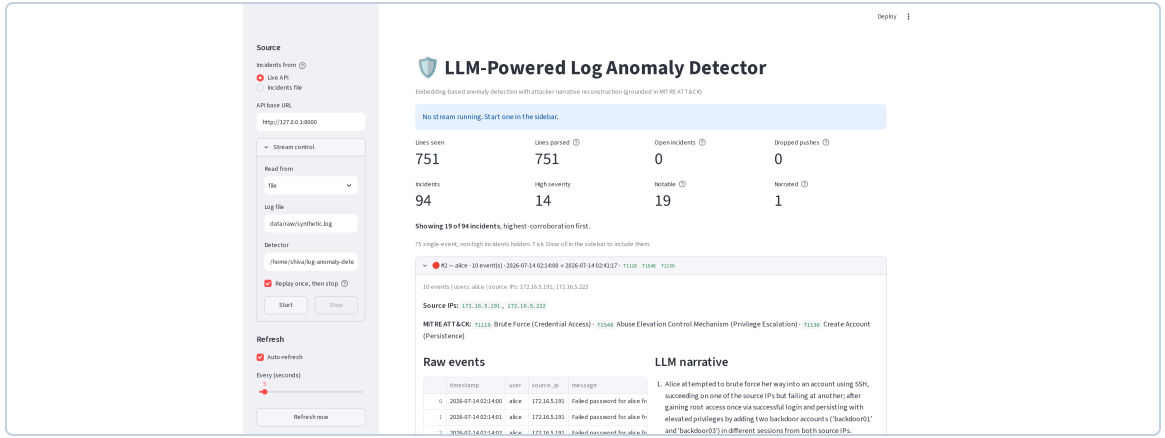
> **Where we are weaker, stated by us:** precision 0.29 — 94 incidents raised for 9 real attacks. Impossible travel is caught by a distance-over-time rule, not by the embeddings. IP rotation, slowing down and the realistic combination all beat us and not Wazuh. Fixing precision is the next stage of work.

Working system

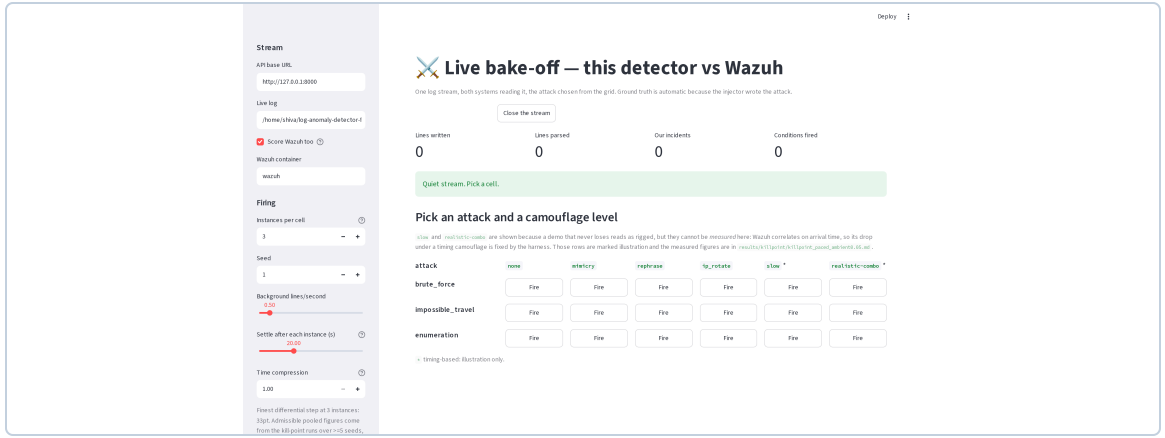
- Replaying the test corpus: **751 lines seen, 751 parsed, 94 incidents**, 19 notable.
- Quiet baseline, no attack at all: **we flag 5 of 30 lines, Wazuh flags 19 of 30** — every one a false positive, on both sides.
- Live bake-off, enumeration: both systems 3 of 3 instances, 100% of the attack's lines.



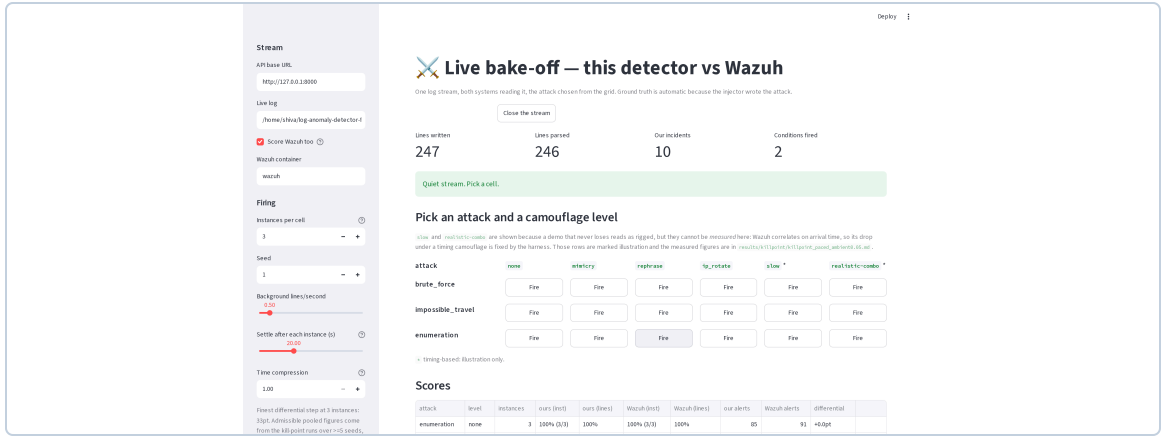
Live dashboard: incidents as the stream emits them, with stream health



One incident: events, severity, MITRE technique, LLM narrative



The bake-off grid: the examiner picks attack and camouflage level



Both systems scored live on the same firing

Technologies and Resources

Software

- Python 3.11+, managed with `uv`
- `sentence-transformers` (all-MiniLM-L6-v2) — turns a log line into a vector of its meaning
- `scikit-learn` Isolation Forest — flags the unusual ones, one model per user
- **Ollama + phi3** — local LLM for the written explanation. No cloud, no API key
- **Wazuh 4.9 manager in Docker** — the third-party SIEM we are compared against
- FastAPI (service and live feed) · Streamlit (dashboard) · pytest (436 tests) · GitHub Actions

Data

- LogHub **OpenSSH** (655,147 real lines, contains real internet attacks) — primary
- LogHub **Linux** (parses at 99.4% after our parser fix) — companion
- 751-line synthetic corpus with full ground truth — regression testing

Hardware

- Minimum, and fully supported: any laptop, **no GPU**, 8 GB RAM
- Optional: NVIDIA GPU, which only speeds up the written explanation (5.2 s vs 24.2 s)
- Demonstration setup: two laptops, one attacking and running the SIEM, one detecting

Project Schedule

Task	Month 1	Month 2	Month 3	Month 4
Literature review	✓			
Requirement analysis	✓	✓		
Design		✓		
Development		✓		
Testing		✓		
Documentation		✓		

Completed so far: foundation and test suite; benchmark platform with a real SIEM; the pre-registered robustness measurement; live streaming, API and dashboard; the live side-by-side bake-off.

Next: sequence modelling to fix precision, then an entity graph for correlation, then attacks generated by real tools on the Kali machine.

References

All eight are journal papers; the PDFs are attached with this presentation (`references/`, and `combined-references.pdf` for all of them in one file).

1. C. Almodovar, F. Sabrina, S. Karimi and S. Azad, "LogFiT: Log anomaly detection using fine-tuned language models," *IEEE Trans. Network and Service Management*, 21(2), 1715–1723, 2024. doi:10.1109/TNSM.2024.3358730
2. M. Landauer, S. Onder, F. Skopik and M. Wurzenberger, "Deep learning for anomaly detection in log data: A survey," *Machine Learning with Applications*, 12, 100470,
3. doi:10.1016/j.mlwa.2023.100470
4. S. Ali, C. Boufaied, D. Bianculli, P. Branco and L. Briand, "A comprehensive study of machine learning techniques for log-based anomaly detection," *Empirical Software Engineering*, 30(5), 129, 2025. doi:10.1007/s10664-025-10669-3
5. X. Wu, H. Li and F. Khomh, "On the effectiveness of log representation for log-based anomaly detection," *Empirical Software Engineering*, 28(6), 137, 2023. doi:10.1007/s10664-023-10364-1
6. Z. Khan, D. Shin, D. Bianculli and L. Briand, "Impact of log parsing on deep learning-based anomaly detection," *Empirical Software Engineering*, 29(6), 139, 2024. doi:10.1007/s10664-024-10533-w
7. G. Pang, C. Shen, L. Cao and A. van den Hengel, "Deep learning for anomaly detection: A review," *ACM Computing Surveys*, 54(2), 1–38, 2021. doi:10.1145/3439950
8. H. Kheddar, "Transformers and large language models for efficient intrusion detection systems: A comprehensive survey," *Information Fusion*, 124, 103347, 2025. doi:10.1016/j.inffus.2025.103347
9. I. Debicha, R. Bauwens, T. Debatty, J.-M. Dricot, T. Kenaza and W. Mees, "TAD: Transfer learning-based multi-adversarial detection of evasion attacks against network intrusion detection systems," *Future Generation Computer Systems*, 138, 185–197, 2023. doi:10.1016/j.future.2022.08.011

Thank you

Questions